

FACE RECOGNITION BASED ATTENDANCE SYSTEM

A PROJECT REPORT

Submitted

*In the partial fulfillment of the requirements for
the award of the degree of*

Bachelor of Technology

in

Electronics and Communication Engineering

By

Ch.Murali Krishna

[Reg. No. 191FA05015]

Y.Jaswanth

[Reg. No. 191FA05068]

M.Lalith Krishna

[Reg. No. 201LA05001]

M.Ganesh

[Reg. No. 191FA05039]

K.Naga Poojitha

[Reg. No. 191FA05241]

Under the guidance of

Dr.K.Annapurna

Associate Professor



(ACCREDITED BY **NAAC** WITH 'A⁺' GRADE)

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**

**VIGNAN'S FOUNDATION FOR SCIENCE, TECHNOLOGY AND
RESEARCH**

(Deemed to be University)

VADLAMUDI, GUNTUR – 522 213, INDIA May – 2023

CERTIFICATE

This is to certify that the project report entitled "**FACE RECOGNITION BASED ATTENDANCE SYSTEM**" that is being submitted by **Ch.Murali Krishna (191FA05015), M.Ganesh (191Fa05039), Y.Jaswanth (191FA05068), K.Naga Poojitha (191FA05241), M.Lalith Krishna (Regd. No. 201LA05001)** in partial fulfilment for the award of B. Tech degree in Electronics and Communication Engineering to Vignan's Foundation for Science Technology and Research University, is a record of bonafide work carried out by them under the supervision of **Dr.K.Annapurna** of ECE Department.



Signature of the faculty guide

Dr.K.Annapurna
Associate Professor



Signature of Head of the Department

Dr.T.Pitchaiah, M.E, Ph.D., MIEEE
Professor

DECLARATION

I hereby declare that the project work entitled "**FACE RECOGNITION BASED ATTENDANCE SYSTEM**" is being submitted to Vignan's Foundation for Science, Technology and Research (Deemed to be University) in partial fulfillment for the award of B. Tech degree in Electronics and Communication Engineering. The work was originally designed and executed by us under the guidance of our supervisor

Dr.K.Annapurna as faculty guide at Department of Electronics and Communication Engineering, Vignan's Foundation for Science Technology and Research (Deemed to be University) and was not a duplication of work done by someone else. I hold the responsibility of the originality of the work incorporated into this thesis.


Signature of the candidate

Ch.Murali Krishna

191FA05015


Signature of the candidate

M.Ganesh

191FA05039


Signature of the candidate

Y.Jaswanth

191FA05068


Signature of the candidate

K.Naga Poojitha

191FA05241


Signature of the candidate

M.Lalith Krishna

201LA05001

ACKNOWLEDGEMENT

We wish to express my deep sense of gratitude to our internal Guide, **Dr.K.Annapurna**, Associate Professor, VFSTR (Deemed to be University) for her able guidance and useful suggestions, which helped us in completing the project work, in time.

We would like to specially thank **Dr.T.Pitchaiah**, Head of the Department, ECE and **Mr.M.Vamsi Krishna** Project Coordinator, Department of ECE, VFSTR (Deemed to be University) for their help and support during the program.

We wish to express our gratitude to **Prof.P.Nagabhushan** Vice-Chancellor, VFSTR (Deemed to be University) for providing me/us the greatest opportunity to have industrial exposure and to carry out the Live Projects.

We would like to thank all my faculty and technical staff of our department for their support to finish my/our work successfully.

We wish to express our heart full thanks to our parents for their support and encouragement throughout our life.

Ch.Murali Krishna(191FA05015)

M.Ganesh (191FA05039)

Y.Jaswanth (191FA05068)

K.Naga Poojitha (191FA05241)

M.Lalith Krishna(201LA05001)

ABSTRACT

In today's environment, taking attendance at schools and colleges is a very significant duty for the teachers. The faculty must manage and keep up with the attendance sheet, which takes time away from lectures. It takes time for the teachers to physically take attendance. The act of physically taking attendance takes a lot of effort and time. Thus, we have been utilising technology. This is crucial for proving how superior the educational system is. The highly developed computerised attendance system can help with attendance management. The use of various biometric methods will be advantageous. Face recognition is the most cutting-edge and well-known technology available today. To address the shortcomings of conventional manual attendance, this study proposes a face recognition-based attendance system. This paper proposes a method for student attendance system in classroom using face recognition technique by using a camera to take pictures of people, uses image processing to extract facial traits, and then compares those features to a database of people who have registered with the system. The attendance is automatically marked if a match is discovered. For face detection and recognition, the Haar-Cascade classifier and the Local Binary Pattern Histogram methods are employed, respectively. In the live-streamed video from the classroom, faces are picked out and acknowledged. The attendance will be sent to the appropriate academics after the session.

Keywords: Haar Cascade Classifier, GUI Interface, OpenCV, Face Detection, Face Recognition, LBPH

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	Page no.
1.1 Project Objective	10
1.2 Background and related work	10
1.3 Feature Extraction	11
1.4 Face Matching	11
1.5 Aims and Objectives	11
1.6 Flow Chart	13
1.7 Disadvantages	14
CHAPTER 2: OVERVIEW OF PROPOSED SYSTEM	
2.1 Definition of terms	16
2.2 Techniques	16
2.3 Digital image processing	18
2.4 Algorithms	19
CHAPTER 3: FACE DETECTION USING HAAR CASCADE CLASSIFIER	
3.1 Introduction	27
3.2 Haar Cascades	27
3.3 Feature Extraction	28
3.4 Haar Feature applied to image	29
3.5 Adaboost	30
3.6 Cascading	31
3.7 The output of face detection	32
CHAPTER 4: USER INTERFACE AND SIMULATION	
4.1 Installed packages	34
4.2 User interface and Frontend Development	37
4.3 Steps for New Registration	39
CHAPTER 5: PROCESS AND RESULTS	
5.1 User Access	42
5.2 Steps for taking attendance	43
5.3 Observed Results	44
CHAPTER 6: PERFORMANCE ANALYSIS AND CONCLUSION	47
REFERENCES	49
APPENDIX	50

LIST OF TABLES

Table	Title	Page
2.1	Advantages & Disadvantages of face detection	20
5.1	Observed attendance results in excel sheets	45
6.1	Measuring of different parameter variations	47

LIST OF FIGURES

Figure	Title	Page no.
1.1	Project Outline	13
2.1	Face recognition using geometric approach	17
2.2	Face images under different lighting condition	17
2.3	Steps in digital image processing	19
2.4	Haar Feature	21
2.5	Integral of Image	21
2.6	LBP Operation	23
2.7	The LPB Operation Radius Change	24
2.8	Extracting the Histogram	24
3.1	Haar Feature extraction kernels	28
3.2	Face detection kernels	28
3.3	Kernels	29
3.4	Detection of face in different parts	30
3.5	Cascading	32
3.6	Detected face	32
4.1	GUI interface	38
4.2	New Registration	39
4.3	For already Registered	39
5.1	User Access for Parents,Students and Professors	42
5.2	Training images stored in a database while student registration	44

CHAPTER-1

INTRODUCTION

1.1 Project Objective:

Attendance is prime important for both the teacher and student of an educational organization. So it is very important to keep record of the attendance. The problem arises when we think about the traditional process of taking attendance in class room.

Calling name or roll number of the student for attendance is not only a problem of time consumption but also it needs energy. So an automatic attendance system can solve all above problems.

There are some automatic attendances making system which are currently used by much institution. One of such system is biometric technique and RFID system. Although it is automatic and a step ahead of traditional method it fails to meet the time constraint. The student has to wait in queue for giving attendance, which is time taking.

This project introduces an involuntary attendance marking system, devoid of any kind of interference with the normal teaching procedure. The system can be also implemented during exam sessions or in other teaching activities where attendance is highly essential. This system eliminates classical student identification such as calling name of the student, or checking respective identification cards of the student, which can not only interfere with the ongoing teaching process, but also can be stressful for students during examination sessions. In addition, the students have to register in the database to be recognized. The enrolment can be done on the spot through the user- friendly interface.

1.2 Background and Related Work:

Much of the work in computer recognition of faces has focused on detecting individual features such as the eyes, mouth, and head outline, and defining the face model by the position, size, and relationships among these features.

Face recognition has become an important issue in many applications such as security systems, credit card verification and criminal identification. For example. the ability to model a particular face and distinguish it from a large number of stored face models would make it possible to vastly improve criminal identification Even the ability to merely detect faces ,as opposed to recognize them, can be important Detecting faces in photographs for automating colour film development can be very useful, since the effect of many enhancement and noise reduction techniques depend on the image content.

Although there are three different approaches to the face recognition problems there are two basic methods from which these three different approaches arise. the first method is based on the information theory concepts, in other words, on the principal component analysis methods. In this approach ,the most relevant information that Fest describes a face is derived from the entire face image, Based on the K arhunen-Loeve expansion in pattern recognition, M.Kirbyand L.Sirovich have shown that any particular face could be economically represented in terms of a best coordinate system that they termed "Eigen faces". These are the Eigen functions of the averaged covariance of the ensemble of faces. Later, M.Turk and A.Pentland have proposed a face recognition method based on the Eigen Faces approach.

The second method is based on extracting feature vectors from the basic parts of a face such as eyes, nose, mouth, and chin. In this method, with the help of deformable templates and extensive mathematics , key information from the basic parts of a face is gathered and then converted into a feature vector.

Face recognition is done in 2 steps:

- I. Feature extraction
2. Face matching

1.3 Feature Extraction:

After the previous two steps, feature extraction is performed resulting in effective information that is useful for distinguishing between faces of different persons and stable with respect to the geometrical and photometrical variations.

1.4 Face Matching:

The extracted features are compared to those stored in the database, and decisions are made according to the sufficient confidence in the match score.

1.5 Aims and Objectives:

The objective of this project is to develop face recognition attendance system.

Expected achievements in order to fulfill the objectives are:

- To detect the face segment from the video frame.
- To extract the useful features from the face detected.
- To classify the features in order to recognize the face detected.
- To record the attendance of the identified student.

1.6 Flow chart:

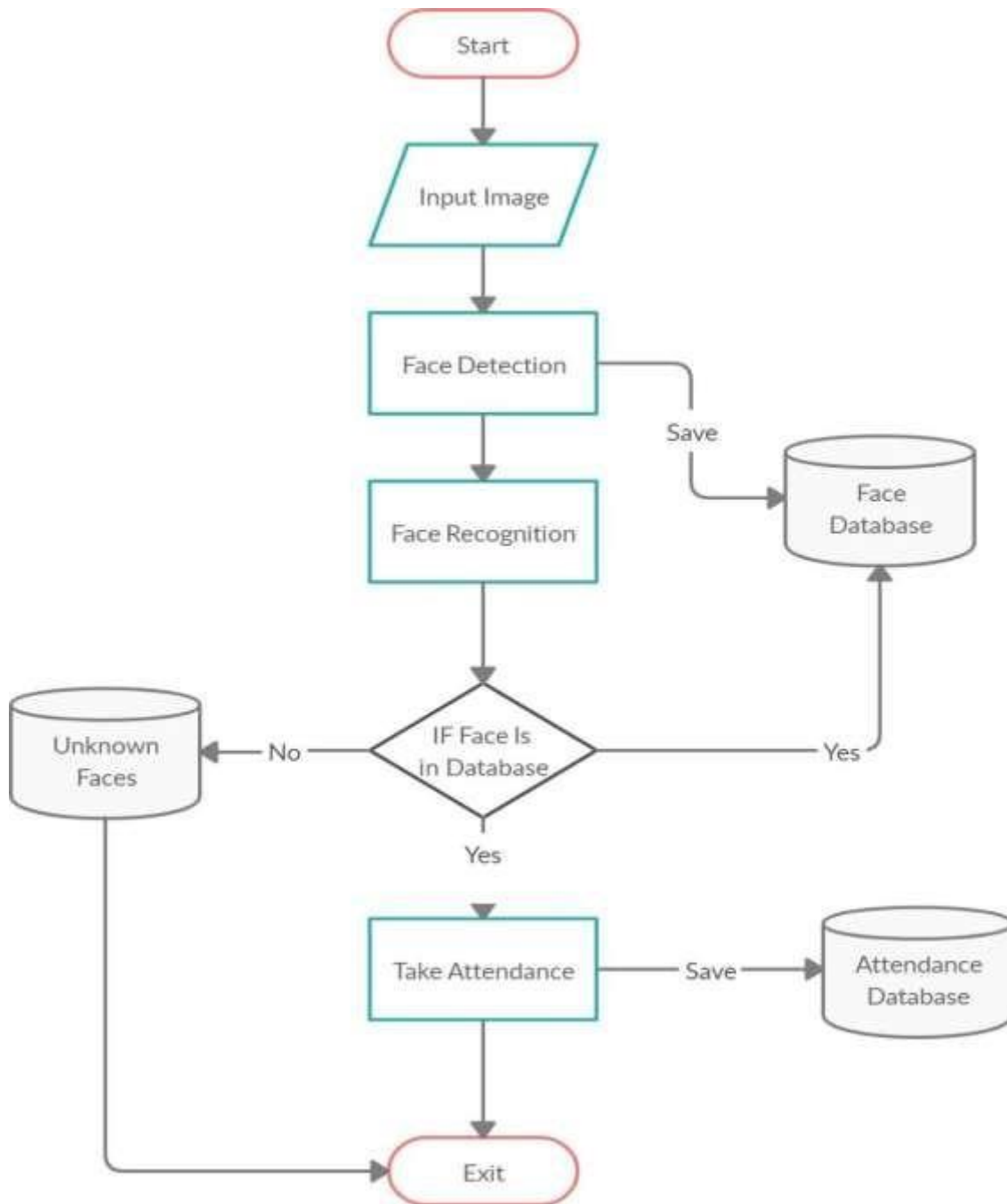


Fig.1.1: Project Outline

1.7 Disadvantages of other technologies for security systems:

1. RFID system
2. Biometric finger print system
3. Password or Pin

1.7.1 Disadvantages of RFID:

1. If a barcode is ripped or damaged there is no way to scan the product.
2. RFID still has two separate chips (read only and readable/writable), which cannot be read by the same machine.

1.7.2 Disadvantages of finger printing system:

1. It can make mistakes with the dryness or dirty of the finger's skin, as well as with the age (is not appropriate with children, because the size of their fingerprint changes quickly).
2. Slow due to one by one process.

1.7.3 Disadvantages of Pin or Password:

1. Can be forgotten.
2. It can be hacked and be seen by a person while the administrator enters the pin or password.

CHAPTER-2

OVERVIEW OF PROPOSED SYSTEM

2.1 Definition of Terms:

2.1.1 Face Recognition:

Face recognition is a pattern recognition task performed on it on be described as classifying a face either "known" or “unknown”, after comparing it with stored individuals, It is also desirable to have system that has the Ability to recognize unknown faces.

2.1.2 Face Detection:

Face detection is the process of identifying and locating all the present faces in a single image or video regardless of their position, scale, orientation, age and expression. Furthermore, the detection should be irrespective of extraneous illumination conditions and the image and video content.

2.2 Techniques:

Some facial recognition algorithms identify facial features by extracting landmarks, or features, from an image of the subject's face. For example, an algorithm may analyze the relative position, size, and/or shape of the eyes, nose, cheekbones, and jaw. These features are then used to search for other images with matching features.

Recognition algorithms can be divided into two main approaches:

a. Geometric approach

This is the historical way to recognize people, geometric features may be generated by segments, perimeters, and areas of some figures formed by the points as shown in fig.2.1. The featured set is studied to compare the recognition result. Distances in the feature space from a template image to every image in the database were calculated. Following to the FERET protocol, 5 nearest face images were derived and if there were photos of the query person then the result was considered positive.

Each image was tested as a query and compared with others. The approach was robust, but its main problem is automatic point location . Some problem arises if image is of bad quality or several points are covered by hair.

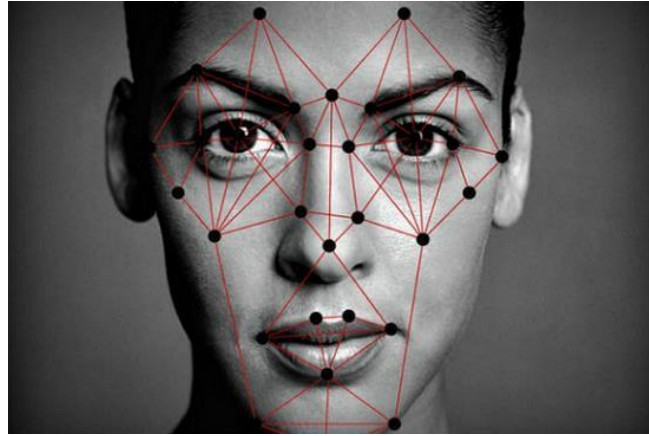


Fig 2.1 : face recognition using geometric approach

b. Photometric approach

It is a statistical approach that distills an image into values and compares the values with templates to eliminate variances. It relies on the image in the presence of light and the geometric location of different angles as shown in Fig.2.2. The photometric transformation is implemented on the source image, does not take into account photometric changes, i.e., changes in the pixel. The main restriction in this approach is that multiple registered images of the same person are required.

Since it recognizes the new image by checking that it is spanned in a linear subspace of the multiple gallery images, it cannot handle the new images of a different person which is not included in the gallery set.



Fig 2.2: Face images under different lightening conditions

2.3 Digital Image Processing:

Digital Image Processing is the processing of images which are digital in nature by a digital computer. Digital image processing techniques are motivated by three major applications mainly:

- Improvement of pictorial information for human perception
- Image processing for autonomous machine application
- Efficient storage and transmission.

2.3.1 Image Representation in a Digital Computer:

An image is a 2-Dimensional light intensity function

$$f(x,y) = r(x,y) \times i(x,y) \quad (2.0)$$

Where, $r(x, y)$ is the reflectivity of the surface of the corresponding image point. $i(x,y)$ Represents the intensity of the incident light. A digital image $f(x, y)$ is discretized both in spatial co-ordinates by grids and in brightness by quantization. Effectively, the image can be represented as a matrix whose row, column indices specify a point in the image and the element value identifies gray level value at that point. These elements are referred to as pixels or pels.

Typically following image processing applications, the image size which is used is **256 × 256**, elements, **640 × 480** pels or **1024 × 1024** pixels. Quantization of these matrix pixels is done at 8 bits for black and white images and 24 bits for colored images (because of the three color planes Red, Green and Blue each at 8 bits).

2.3.2 Steps in Digital Image Processing:

Digital image processing involves the following basic tasks from the below Fig.2.3:

- Image Acquisition - An imaging sensor and the capability to digitize the signal produced by the sensor.
- Preprocessing – Enhances the image quality, filtering, contrast enhancement etc.
- Segmentation – Partitions an input image into constituent parts of objects.
- Description/feature Selection – extracts the description of image objects suitable for

further computer processing.

- Recognition and Interpretation – Assigning a label to the object based on the information provided by its descriptor. Interpretation assigns meaning to a set of labelled objects.
- Knowledge Base – This helps for efficient processing as well as inter module cooperation.

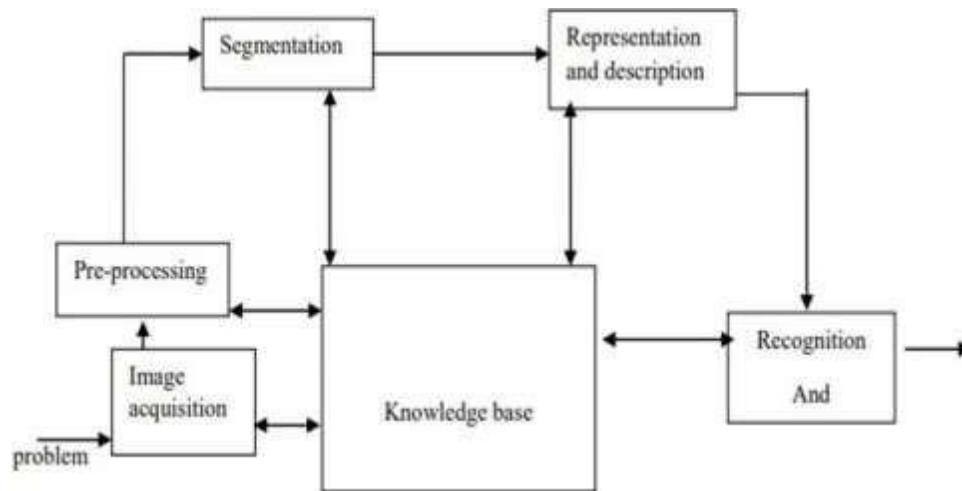


Fig. 2.3 : A diagram showing the steps in digital image processing

2.4 Algorithms:

Face recognition is a technology that can identify the face of an individual whose image is stored in a dataset. Although other identification methods may be more accurate, facial recognition has been an important focus of research because it is easy to implement, convenient, and non-obtrusive.

A face recognition algorithm is a basic component of a face detection and recognition system. Face recognition algorithms typically perform the following main tasks:

- Detect faces in images, videos or live streams
- Compute a mathematical model of the face image
- Compare the model derived from a face to an image in a training set or database.

- Evaluate the comparison to see whether the face shows the required individual

A face Detector has to tell whether an image of arbitrary size contains a human face and if so, where it is. Face detection can be performed based on several cues: skin color (for faces in color images and videos, motion (for faces in videos), facial/head shape, facial appearance or a combination of these parameters. Most face detection algorithms are appearance based without using other cues. An input image is scanned at all possible locations and scales by a sub window. Face detection is posed as classifying the pattern in the sub window either as a face or a non-face. The face/nonface classifier is learned from face and non-face training examples using statistical learning methods[9]. Most modern algorithms are based on the Viola Jones object detection framework, which is based on Haar Cascades.

Table 2.1: Advantages & Disadvantages of Face Detection Methods

Face Detection Method	Advantages	Disadvantages
Viola Jones Algorithm	1 . High detection Speed. 2 . High Accuracy.	1. Long Training Time. 2.Limited Head Pose. 3.Not able to detect darkfaces.
Local Binary Pattern Histogram	1.Simple computation. 2.High tolerance against the monotonic illumination changes.	1.Only used for binary and grey images. 2.Overall performance is inaccurate compared to Viola-JonesAlgorithm.
Ada Boost Algorithm	Need not to have any prior knowledge about face structure.	The result highly depends on the training data and affected by weak classifiers.
SMQT Features and SNOW Classifier Method	1 . Capable to deal with lighting problem in object detection. 2 . Efficient in computation.	The region contain very similar to grey value regions will be misidentified as face.
Neural-Network	High accuracy only if large size of image were trained.	1 . Detection process is slow and computation is complex. 2 . Overall performance is weaker than Viola-Jones algorithm.

2.4.1 Viola-Jones Algorithm:

Viola-Jones algorithm which was introduced by P. Viola, M. J. Jones (2001) is the most popular algorithm to localize the face segment from static images or video frame. Basically the concept of Viola-Jones algorithm consists of four parts shown in Fig.2.4. The first part is known as Haar feature, second part is where integral image is created, followed by implementation of Adaboost on the third part and lastly cascading process.



Fig. 2.4: Haar Feature

Viola-Jones algorithm analyses a given image using Haar features consisting of multiple rectangles. In the fig shows several types of Haar features. The features perform as window function mapping onto the image. A single value result, which representing each feature can be computed by subtracting the sum of the white rectangle(s) from the sum of the black rectangle(s) shown in Fig2.5.

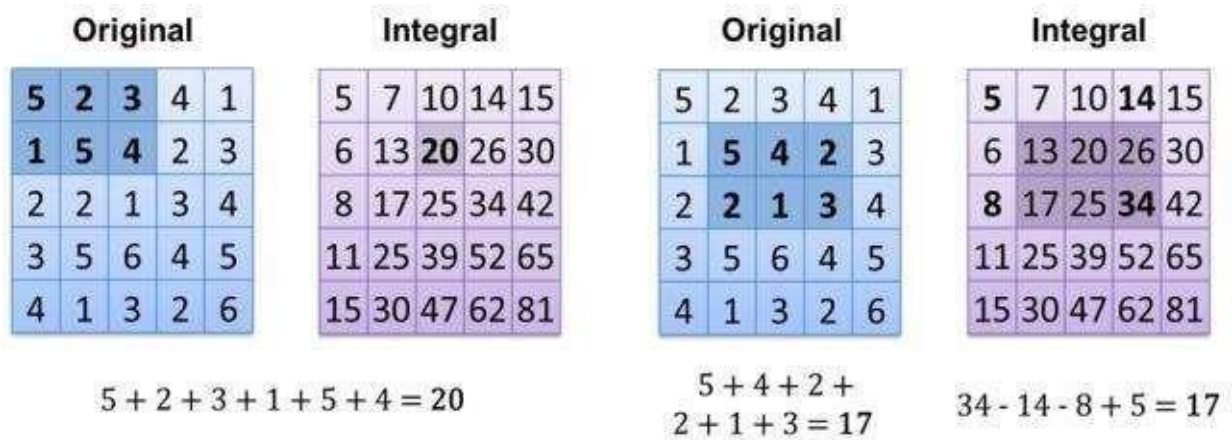


Fig. 2.5: Integral of Image

The value of integrating image in a specific location is the sum of pixels on the left and the top of the respective location. In order to illustrate clearly, the value of the integral image at location 1 is the sum of the pixels in rectangle A. The values of integral image at the rest of the locations are cumulative. For instance, the value at location 2 is summation of A and B, $(A + B)$, at location 3 is summation of A and C, $(A + C)$, and at location 4 is summation of all the regions, $(A + B + C + D)$. Therefore, the sum within the D region can be computed with only addition and subtraction of diagonal at location $4 + 1 - (2 + 3)$ to eliminate rectangles A, B and C.

2.4.2 Local Binary Pattern Histogram:

Local Binary Pattern (LBP) is a simple yet very efficient texture operator which labels the pixels of an image by thresholding the neighborhood of each pixel and considers the result as a binary number. It was first described in 1994 (LBP) and has since been found to be a powerful feature for texture classification. It has further been determined that when LBP is combined with histograms of oriented gradients (HOG) descriptor, it improves the detection performance considerably on some datasets. Using the LBP combined with histograms we can represent the face images with a simple data vector.

LBPH algorithm work step by step:

LBPH algorithm work in 5 steps.

1. **Parameters:** the LBPH uses 4 parameters:

- **Radius:** the radius is used to build the circular local binary pattern and represents the radius around the central pixel. It is usually set to 1.
- **Neighbors:** the number of sample points to build the circular local binary pattern. Keep in mind: the more sample points you include, the higher the computational cost. It is usually set to 8.
- **Grid X:** the number of cells in the horizontal direction. The more cells, the finer the grid, the higher the dimensionality of the resulting feature vector. It is usually set to 8.

- **Grid Y:** the number of cells in the vertical direction. The more cells, the finer the grid, the higher the dimensionality of the resulting feature vector. It is usually set to 8.
2. **Training the Algorithm:** First, we need to train the algorithm. To do so, we need to use a dataset with the facial images of the people we want to recognize. We need to also set an ID (it may be a number or the name of the person) for each image, so the algorithm will use this information to recognize an input image and give you an output. Images of the same person must have the same ID. With the training set already constructed, let's see the LBPH computational steps.
 3. **Applying the LBP operation:** The first computational step of the LBPH is to create an intermediate image that describes the original image in a better way, by highlighting the facial characteristics. To do so, the algorithm uses a concept of a sliding window, based on the parameters radius and neighbors.

The image below shows this procedure:

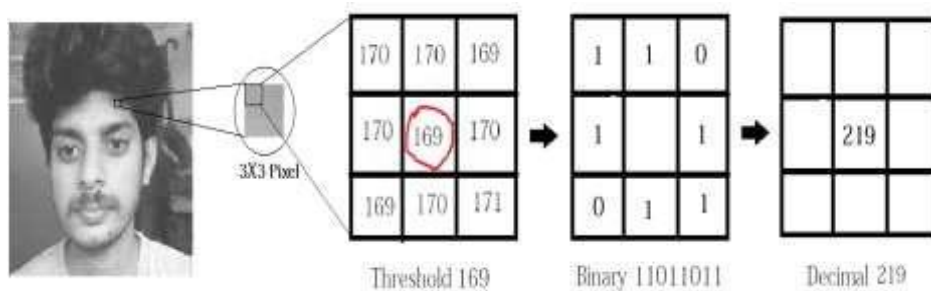


Fig.2.6: LBP Operation

Based on the Fig.2.6 above, let's break it into several small steps so we can understand it easily:

- Suppose we have a facial image in grayscale.
- We can get part of this image as a window of 3x3 pixels.
- It can also be represented as a 3x3 matrix containing the intensity of each pixel (0~255).
- Then, we need to take the central value of the matrix to be used as the threshold.
- This value will be used to define the new values from the 8 neighbors.
- For each neighbor of the central value (threshold), we set a new binary value. We set 1 for values equal or higher than the threshold and 0 for values lower than the threshold.
- Now, the matrix will contain only binary values (ignoring the central value). We need to concatenate each binary value from each position from the matrix line by line into a new binary

value (e.g. 10001101). Note: some authors use other approaches to concatenate the binary values (e.g. clockwise direction), but the final result will be the same.

- Then, we convert this binary value to a decimal value and set it to the central value of the matrix, which is actually a pixel from the original image.
- At the end of this procedure (LBP procedure), we have a new image which represents better the characteristics of the original image.

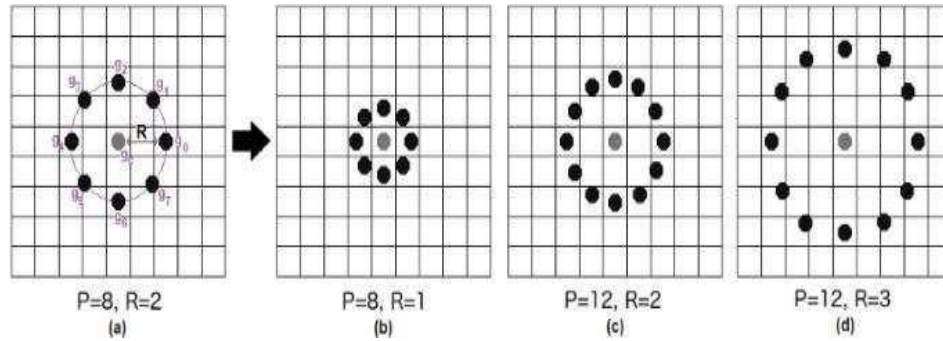


Fig.2.7: The LBP operation Radius Change

It can be done by using bilinear interpolation. If some data point is between the pixels, it uses the values from the 4 nearest pixels (2x2) to estimate the value of the new data point from Fig.2.7.

4. **Extracting the Histograms:** Now, using the image generated in the last step, we can use the Grid X and Grid Y parameters to divide the image into multiple grids, as can be seen in the following fig.2.8:

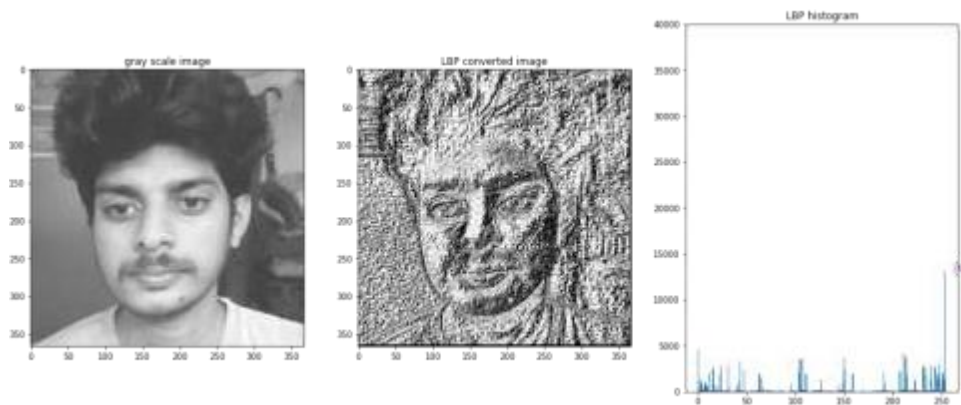


Fig. 2.8: Extracting The Histogram

Based on the image above, we can extract the histogram of each region as follows:

- As we have an image in grayscale, each histogram (from each grid) will contain only 256 positions (0~255) representing the occurrences of each pixel intensity.
- Then, we need to concatenate each histogram to create a new and bigger histogram. Supposing we have 8x8 grids, we will have $8 \times 8 \times 256 = 16.384$ positions in the final histogram. The final histogram represents the characteristics of the image original image.

5 . Performing the face recognition: In this step, the algorithm is already trained. Each histogram created is used to represent each image from the training dataset. So, given an input image, we perform the steps again for this new image and creates a histogram which represents the image.

- So to find the image that matches the input image we just need to compare two histograms and return the image with the closest histogram.
- We can use various approaches to compare the histograms (calculate the distance between two histograms), for example: Euclidean distance, chi-square, absolute value, etc. In this example, we can use the Euclidean distance (which is quite known) based on the following formula:

$$D = \sqrt{\sum_{i=1}^n (hist1_i - hist2_i)^2}$$

- So the algorithm output is the ID from the image with the closest histogram. The algorithm should also return the calculated distance, which can be used as a ‘confidence’ measurement.
- We can then use a threshold and the ‘confidence’ to automatically estimate if the algorithm has correctly recognized the image. We can assume that the algorithm has successfully recognized if the confidence is lower than the threshold defined.

CHAPTER-3

FACE DETECTION USING HAAR CASCADE

CLASSIFIER

3.1 INTRODUCTION:

Face detection is a type of application classified under “computer vision” technology. It is the process in which algorithms are developed and trained to properly locate faces or objects (in object detection, a related system), in images. These can be in real time from a video camera or from photographs. An example where this technology is used are in airport security systems. In order to recognize a face, the camera software must first detect it and identify the features before making an identification. Likewise, when Facebook makes tagging suggestions to identify people in photos it must first locate the face. On social media apps like Snapchat, face detection is required to augment reality which allows users to virtually wear dog face masks using fancy filters. Another use of face detection is in smartphone face ID security.

In this project, I implemented a system for locating faces in digital images. These are in JPEG format only. Before we continue, we must differentiate between face recognition and face detection. They are not the same, but one depends on the other. In this case face recognition needs face detection for making an identification to “recognize” a face. I will only cover face detection.

Face detection uses classifiers, which are algorithms that detects what is either a face(1) or not a face(0) in an image. Classifiers have been trained to detect faces using thousands to millions of images in order to get more accuracy. OpenCV uses two types of classifiers, LBP (Local Binary Pattern) and Haar Cascades.

3.2 HAAR CASCADES:

Haar Cascade classifier is based on the Haar Wavelet technique to analyse pixels in the image into squares by function. This uses “integral image” concepts to compute the “features” detected. Haar Cascades use the Ada-boost learning algorithm which selects a small number of important features from a large set to give an efficient result of classifiers then use cascading techniques to detect face in a image. Here are some Haar-Features shown below Fig 3.1.

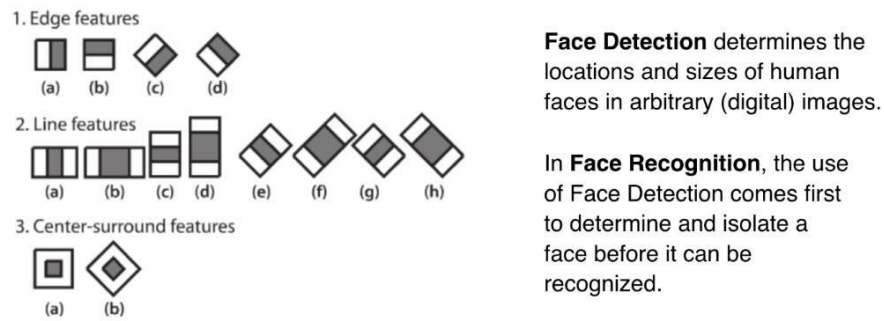


Fig 3.1: Haar feature extraction kerneals.

3.3 FEATURE EXTRACTION:

As I mentioned earlier, Haar Cascades use machine learning techniques in which a function istrained from a lot of positive and negative images. This process in the algorithm is feature extraction from below Fig 3.2.

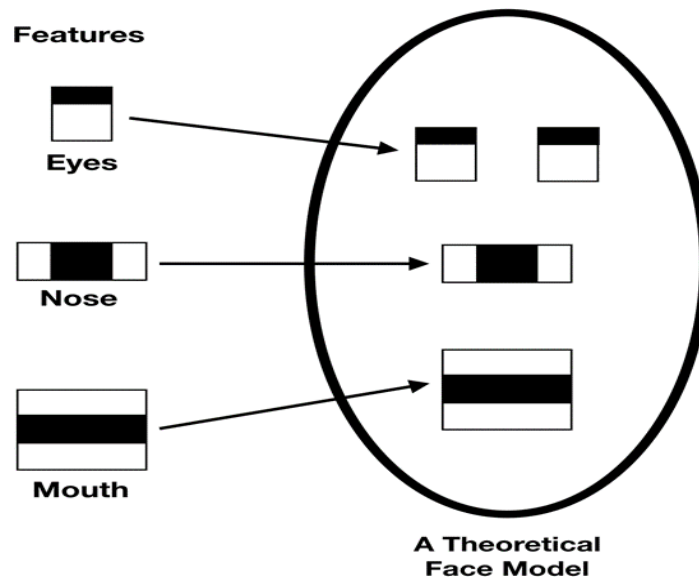


Fig 3.2:Face detection Kernels.

Haar features are similar to these convolution kernels which are used to detect the presence of that feature in the given Fig.3.3. The first two are “edge features”, used to detect edges. The third is a “line feature”, while the fourth is a “four rectangle feature”, most likely used to detect a slanted line.

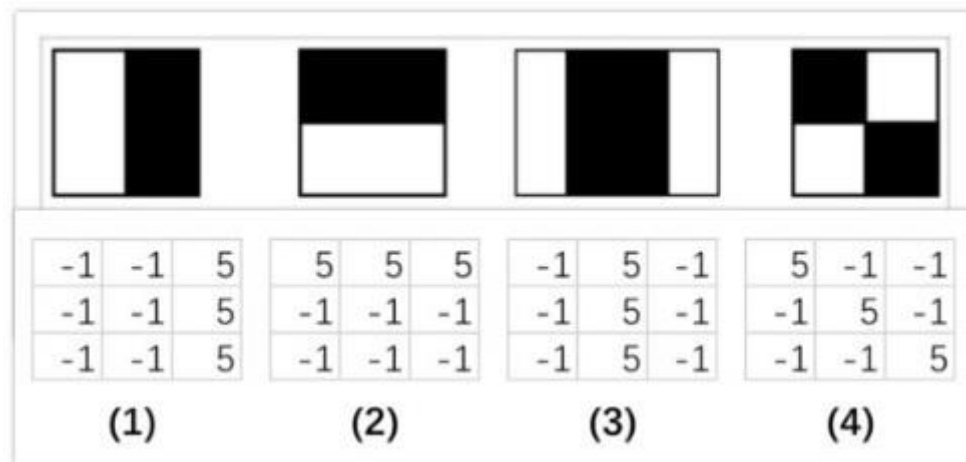


Fig.3.3:Kerneals

3.4 HAAR FEATURES APPLIED TO IMAGE:

Each feature results in a single value which is calculated by subtracting the sum of pixels under white rectangle from the sum of pixels under black rectangle. Every haar feature has some sort of resemblance to identify a part of face shown in below Fig 3.4.

$$\text{Value} = \sum (\text{pixels in white area}) - \sum (\text{pixels in black area})$$

If we consider all possible parameters of the haar features like position, scale and type we end up calculating about 160,000+ features. So we need to evaluate huge set of features for every 24*24 PX.

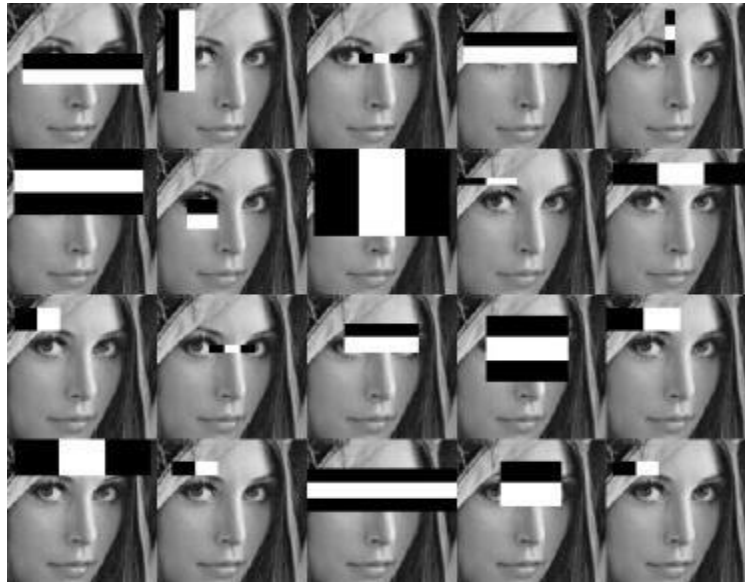


Fig.3.4: Detection of face in different parts

So to avoid this we have a idea to avoid redundant features and pick only those features which are very useful for us. This can be done using AdaBoost.

3.5 ADABOOST:

AdaBoost is a machine learning algorithm that able to find the best features among 160,000+ features. These features are also called as weak classifiers. After these features are found a weighted combination of all these features in used in evaluating and deciding any given window has a face or not. Each of the selected features (weak classifiers) are considered okay to be included if they can at least perform better than random guessing (detects more than half the cases). Each of the weak classifier is relevant detecting a part of face. Output of weak classifier is binary if it have identified a part of face or not.

Adaboost constructs a strong classifier as a linear combination of these weak classifiers.

Strong classifier = linear sum of weak classifiers

$$F(x) = \sum (\alpha_i * f_i(x))$$

here α_i are corresponding weights to each weak classifier $f_i(x)$

Even if there are more face's detected the excessive large amount of evaluated value remains negative. So the algorithm will more concentrate on discarding non faces and spending more time on probable face region. Hence a single strong classifier formed out of linear combination of all best features is not a good to evaluate on each window because of computation cost.

3.6 CASCADING:

We start with simple classifiers which reject many of the negative sub-windows while detecting almost all positive sub-windows. Positive response from the first classifier triggers the evaluation of a second (more complex) classifier, and so on . A negative outcome at any point leads to the immediate rejection of the sub-window.

Given a input image sub-window we need to move our 24*24 window all over the image and compute 2,500 features for every window and take a linear combination of all outputs and see if it exceeds a certain threshold or not.

- The basic principle of the Viola-Jones face detection algorithm is to scan the detector many times through the same image — each time with a new size.
- Even if an image should contain one or more faces it is obvious that an excessive large amount of the evaluated sub-windows will still be negatives (non-faces).
- So the algorithm should concentrate on discarding non-faces quickly and spend more on time on probable face regions.
- Hence a single strong classifier formed out of linear combination of all best features is not a good to evaluate on each window because of computation cost

Instead of calculating 2,500 features for every window we use idea of cascades. We do a sampling of 2,500 features into x different cascades. Now we can detect if there is face or not in different cascades linearly. If cascade_i finds a face in a image then image is passed to next cascade. If no face is found in a cascade we can move to next window. This reduces the time complexity. Process shown below Fig.3.5.

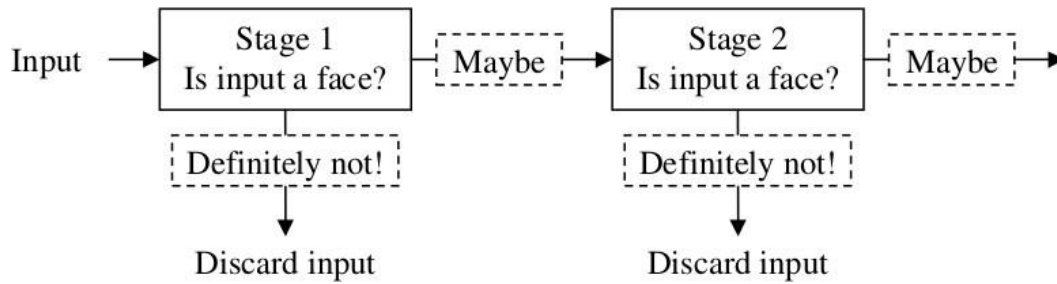


Fig 3.5: Cascading

The job of each stage is used to determine whether a given sub window is definitely not a face or may be a face. A given sub window is immediately discarded as not a face if it fails in any of the stage.

3.7 The output of face detection :

In the Fig.3.6 it is clearly shown in below images the face of student was detected with their names.

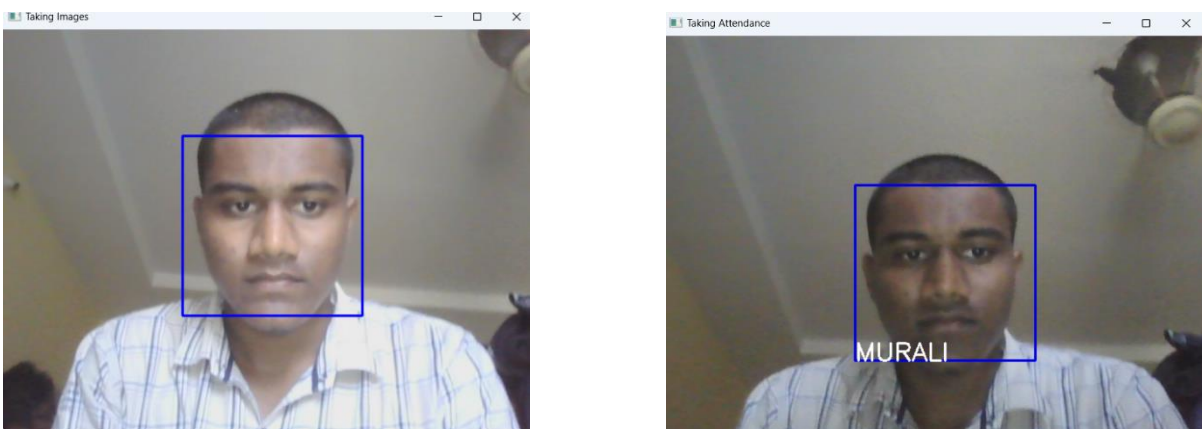


Fig.3.6: Detected Face

CHAPTER-4

USER INTERFACE AND SIMULATION

4.1 INSTALLED PACKAGES:

Face recognition is a field of computer vision that involves identifying and verifying the identity of a person based on their facial features. It has many practical applications, such as security systems, surveillance, and biometric authentication.

There are several steps involved in the face recognition process, including face detection, face alignment, feature extraction, and classification. Face detection involves locating the face in an image or video frame, while face alignment involves normalizing the face to a standard pose and size. Feature extraction involves extracting relevant features from the face, such as the location of facial landmarks or the texture of the skin. Finally, classification involves using a machine learning algorithm to match the extracted features to a database of known faces.

There are many tools and libraries available for face recognition, including OpenCV, dlib, and TensorFlow. These tools provide a wide range of functionality, from basic face detection to more advanced tasks like facial landmark detection and face clustering.

Here are the some useful packages needed for the frontend development and face recognition and detection purposes.

- Tk-tools
- OpenCV
- Pytest-shutil
- Python-csv
- Numpy
- Pandas

4.1.1 Tkinter:

Tkinter is a Python library that provides a simple way to create graphical user interfaces (GUIs) for Python applications. It is a standard part of the Python language and is included with most Python installations. Tkinter provides a set of widgets, such as buttons, labels, and text boxes, that can be used to create a GUI. These widgets can be arranged in a variety of ways to create complex layouts. Tkinter also provides a set of event handlers that can be used to respond to user input, such as button clicks or key presses.

One of the advantages of Tkinter is its simplicity. It is easy to learn and use, even for beginners. Tkinter also has good documentation and a large community of users, which makes it easy to find help and examples online.

Tkinter is not limited to creating GUIs for face recognition applications. It can be used to create GUIs for a wide range of Python applications, from simple scripts to complex desktop applications. Tkinter is also cross-platform, which means that GUIs created with Tkinter will work on Windows, macOS, and Linux.

Overall, Tkinter is a powerful and versatile library that can be used to create GUIs for a wide range of Python applications. Its simplicity and ease of use make it a popular choice for beginners and experienced developers alike.

4.1.2 Open CV:

OpenCV (Open Source Computer Vision) is a popular open-source library for computer vision and image processing. It provides a wide range of functions and algorithms for image and video processing, including face detection and recognition.

OpenCV provides several pre-trained face detection models, such as Haar cascades and deep learning-based models. These models can be used to detect faces in images and video streams. Once a face is detected, OpenCV provides functions for face alignment, feature extraction, and classification.

Face alignment involves normalizing the face to a standard pose and size. OpenCV provides functions for detecting facial landmarks, such as the location of the eyes, nose, and mouth, which can be used to align the face. Feature extraction involves extracting relevant features from the face, such as the texture of the skin or the location of facial landmarks. OpenCV provides functions for extracting these features, which can be used for face recognition.

OpenCV also provides several face recognition algorithms, such as Eigenfaces, Fisherfaces, and Local Binary Patterns Histograms (LBPH). These algorithms can be used to match the extracted features to a database of known faces.

One of the advantages of OpenCV is its speed and efficiency. It is written in C++ and optimized for performance, which makes it suitable for real-time face detection and recognition applications. OpenCV also has good documentation and a large community of users, which makes it easy to find help and examples online.

Overall, OpenCV is a powerful and versatile library for face detection and recognition. Its speed and efficiency make it suitable for real-time applications, while its wide range of functions and algorithms make it suitable for a variety of face recognition tasks.

4.1.3 NumPy:

NumPy is a popular Python library for numerical computing. It provides a powerful array data structure and a wide range of functions for manipulating arrays, such as mathematical operations, indexing, and slicing. NumPy is often used in conjunction with other libraries for scientific computing, such as OpenCV, for face recognition and detection.

NumPy can be used for a variety of tasks in face recognition and detection. For example, it can be used to preprocess images before they are fed into a face detection algorithm. This might involve converting the image to grayscale, resizing it to a standard size, or normalizing the pixel values.

NumPy can also be used for feature extraction in face recognition. For example, it can be used to extract features from the face, such as the texture of the skin or the location of facial landmarks. These features can then be used to train a machine learning algorithm for face recognition.

In addition, NumPy can be used for data manipulation and analysis in face recognition. For example, it can be used to store and manipulate the data used for training a face recognition algorithm, such as the images of known faces and their corresponding labels.

One of the advantages of NumPy is its speed and efficiency. It is written in C and optimized for performance, which makes it suitable for large-scale numerical computations. NumPy also has good documentation and a large community of users, which makes it easy to find help and examples online.

Overall, NumPy is a powerful and versatile library for numerical computing, and it can be used for a variety of tasks in face recognition and detection. Its speed and efficiency make it suitable for large-scale computations, while its wide range of functions make it suitable for a variety of face recognition tasks.

4.1.4 Pandas:

Pandas is a popular Python library for data manipulation and analysis. It provides a powerful data structure called a DataFrame, which is similar to a table in a relational database. Pandas is often used in conjunction with other libraries for scientific computing, such as NumPy and OpenCV, for face recognition and detection.

Pandas can be used for a variety of tasks in face recognition and detection. For example, it can be used to store and manipulate the data used for training a face recognition algorithm, such as the images of known faces and their corresponding labels. The data can be loaded into a Pandas DataFrame, which provides a convenient way to manipulate and analyze the data.

Pandas can also be used for data preprocessing in face recognition. For example, it can be used to clean and transform the data before it is fed into a face recognition algorithm. This might involve removing missing values, scaling the data, or encoding categorical variables.

In addition, Pandas can be used for data visualization in face recognition. For example, it can be used to create plots and charts that help to visualize the data and the results of a face recognition algorithm.

One of the advantages of Pandas is its flexibility and ease of use. It provides a wide range of functions for data manipulation and analysis, and it is easy to learn and use, even for beginners. Pandas also has good documentation and a large community of users, which makes it easy to find help and examples online.

Overall, Pandas is a powerful and versatile library for data manipulation and analysis, and it can be used for a variety of tasks in face recognition and detection. Its flexibility and ease of use make it suitable for a wide range of face recognition applications.

4.2 User Interface and Frontend Development:

From below Fig.4.1, the GUI interface is the visual component of the system that allows users to interact with the software. It typically includes buttons, menus, and other graphical elements that users can click on or interact with to perform various functions.

In the case of a face recognition based attendance system, the GUI interface might include options for adding new users to the database, viewing attendance records, and generating reports. For example, the system might allow an administrator to add new employees to the database by uploading a photo of their face and entering their name and other relevant information.

The GUI interface might also include features such as real-time monitoring of attendance, notifications for late or absent employees, and integration with other systems such as payroll or HR software. For example, the system might send an email or text message to an administrator if an employee is late or absent, or it might automatically update the employee's attendance record in the payroll system.

Another important aspect of the GUI interface is its ease of use. The interface should be intuitive and user-friendly, with clear instructions and easy-to-understand icons and buttons. The system should also be

customizable to meet the specific needs of the organization, such as the ability to add custom fields or modify the layout of the interface.

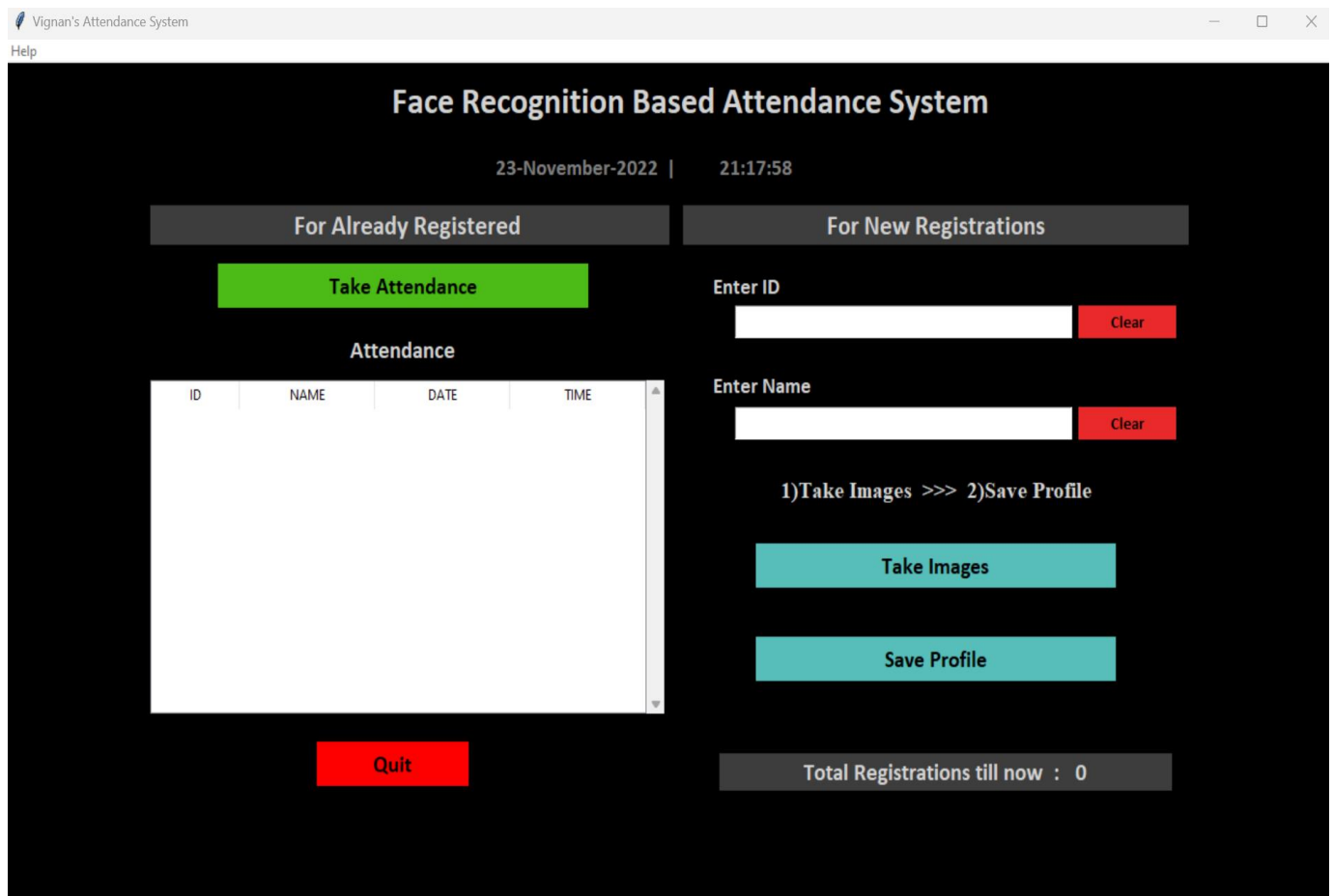


Fig.4.1: GUI Interface

The GUI interface is designed in a way that each user should be register by using their institution ID and their respective Name,

Hence after the data entry process the user have to give their identity of face to the camera by Take Images to proceed. Then the data base is used to store the user images of different angles in the count of 100 to 500

images per 25 to 30 seconds. Based on our requirements we can change the count of images taken and duration of taking images.

As many of the images given to the database or taken and stored in database that much accurately the algorithm can detect the identity of the person/target.

Fig. 4.2: New Registration

Fig.4.3: For Already Registered

4.3 Steps for New Registration:

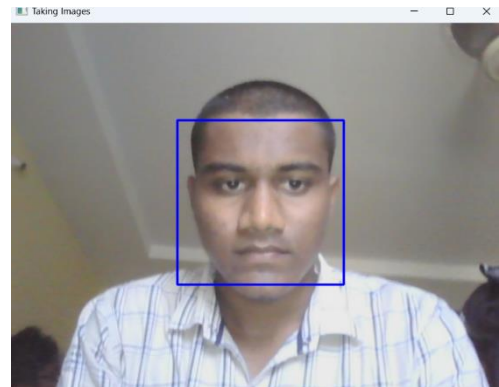
As shown in above ig.4.2 the process shown below,

1. Select New Registration Form

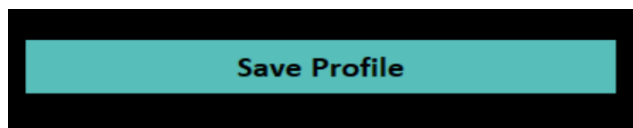
2. Enter Student Details



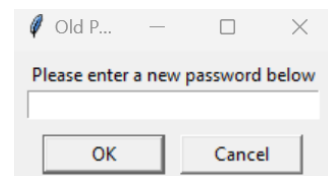
3. Click Take Images Button



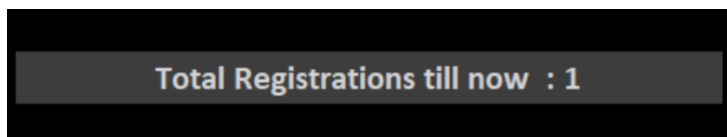
4. Scan the Face in Different Angles



5. Click on Save Profile



6. Enter the Password



7. Then we can see Registered No.of Students

CHAPTER-5

PROCESS AND RESULTS

5.1 USER ACCESS:

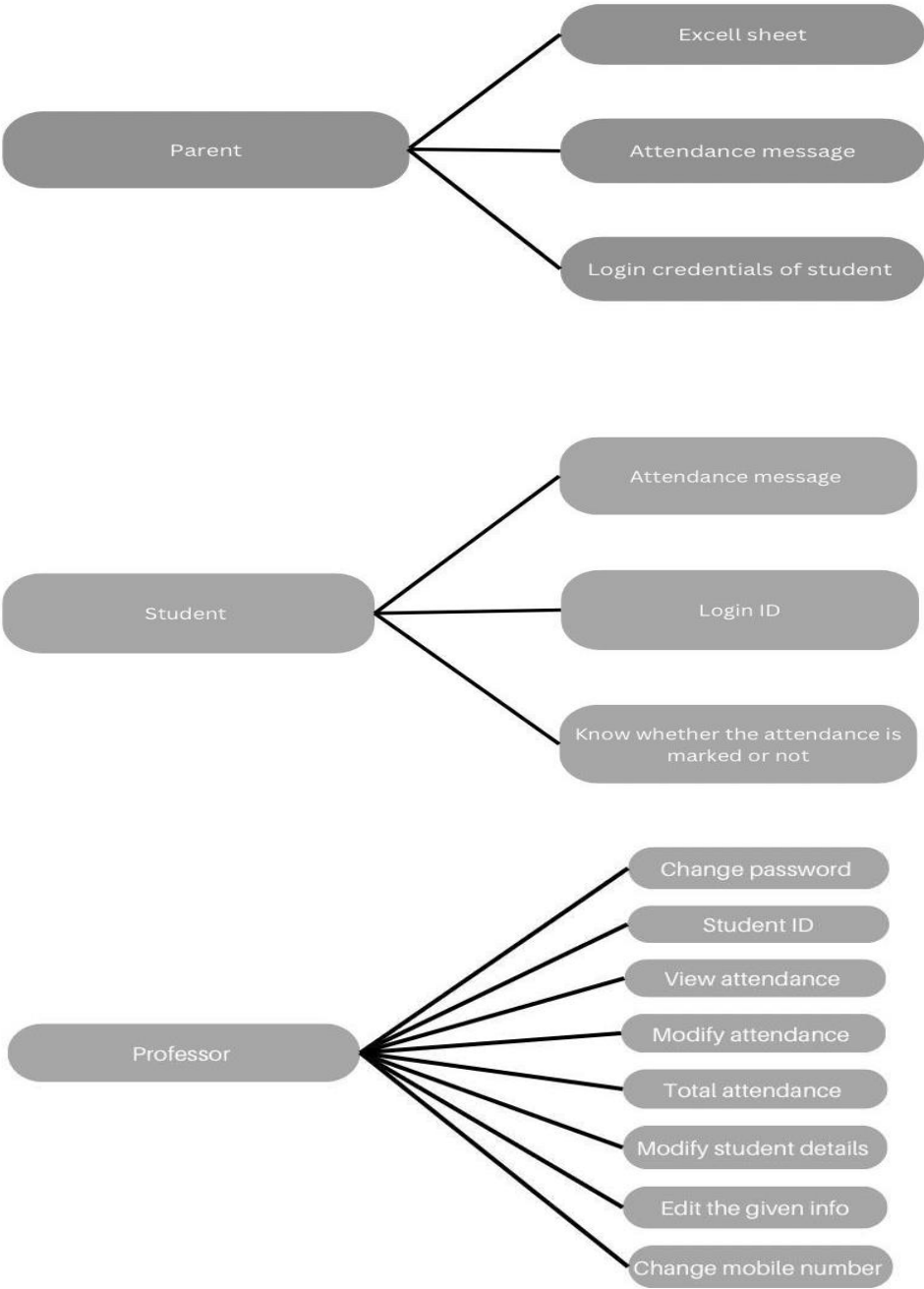
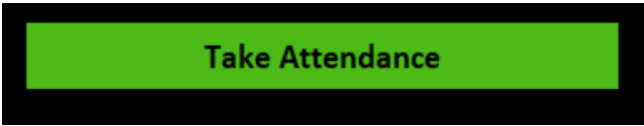


Fig.5.1:User Access for Parents, Students and Professors

5.2 STEPS FOR TAKING ATTENDANCE:



1. Click on Take Attendance Button



2. Stay in the Radius of your web cam

ID	NAME	DATE	TIME
5015	CH Murali	23-11-2022	21:24:37

3. Check the Status of your Attendance



4. Press Quit for Stop Taking Attendance

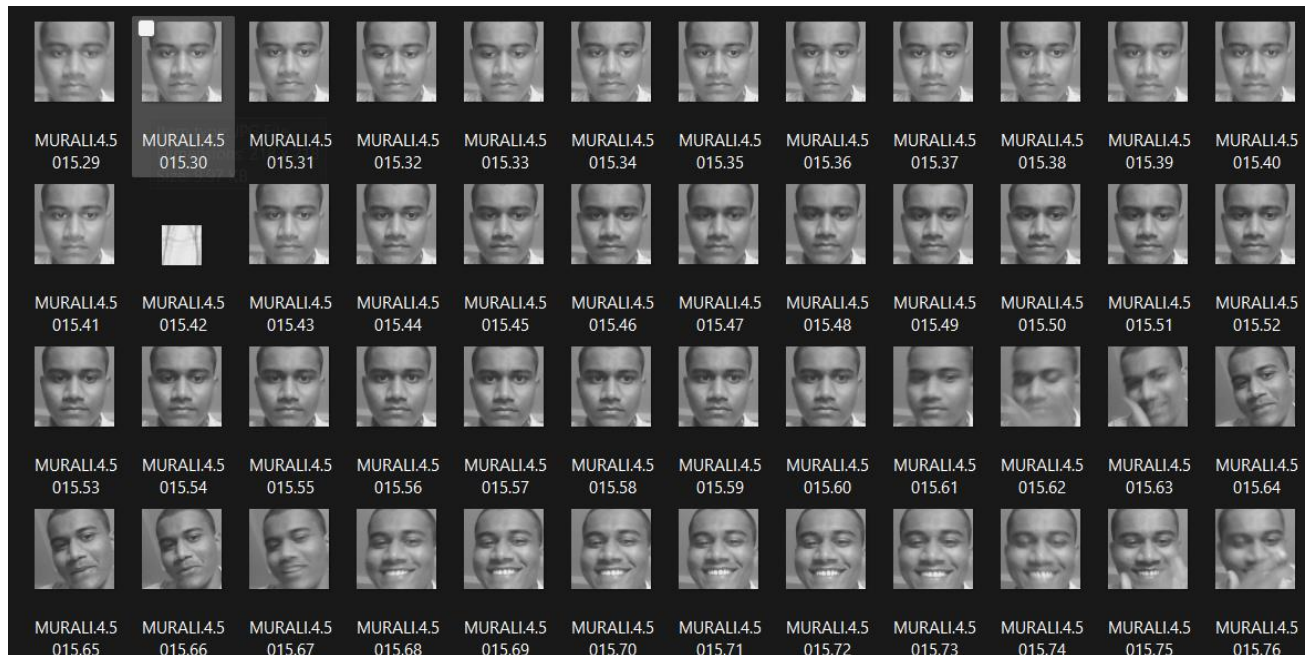
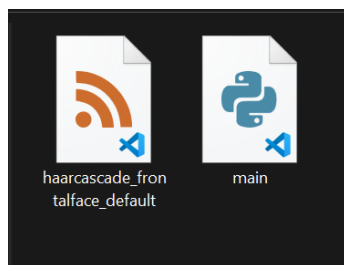
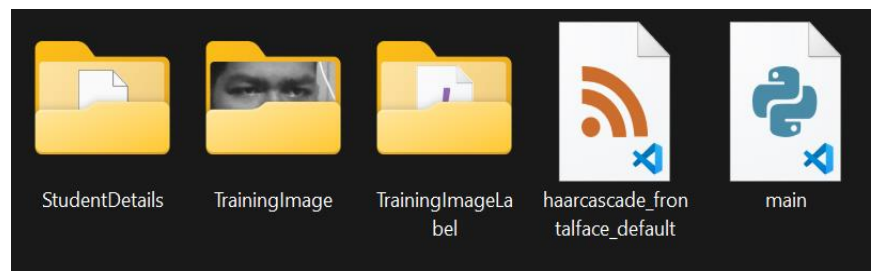


Fig. 5.2: Training Images Stored in Database while student registration

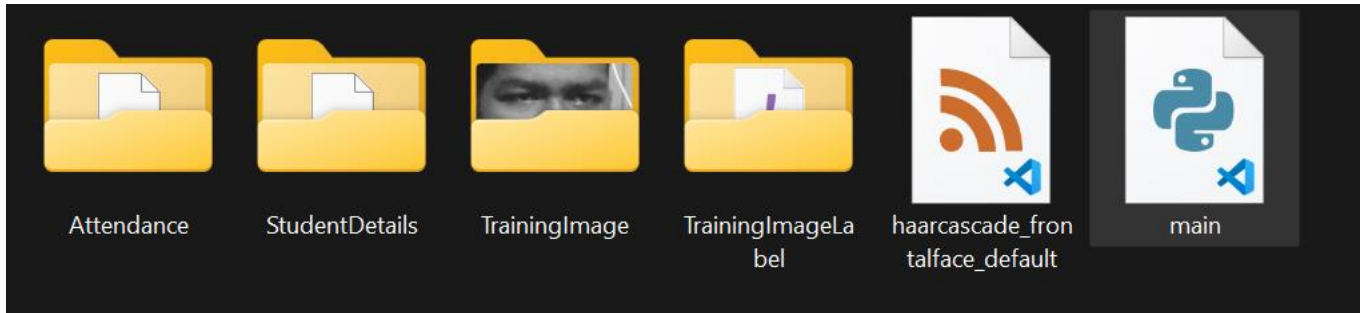
5.3 OBSERVED RESULTS:



1. Files Before Student Registration



2. Added Training Images and Student details after Registration



3.Attendance Report (Excel file) added after Taking Attendance

Table 5.1 Observed Attendance Result in Excel Sheets

Id	Name	Date	Time
5015	Muralli	03/28/23-03-2023	11:57:55
5015	Muralli	03/28/23-03-2023	11:58:10
5039	Ganesh	03/28/23-03-2023	11:58:18
5039	Ganesh	03/28/23-03-2023	11:58:34

CHAPTER-6

PERFORMANCE ANALYSIS AND CONCLUSION

Face Recognition based Attendance System using Haar Cascade algorithm and Excel sheets to get the data of attendance is a promising development in the field of attendance tracking. The use of Haar Cascade algorithm for face detection is a well-established technique that has been used in many successful face recognition systems. The use of Excel sheets to store attendance data is also a practical and efficient solution.

In terms of performance observed in below table 6.1, the system is able to accurately recognize faces and record attendance in real-time. The use of Haar Cascade algorithm ensures that the system is able to detect faces even in challenging lighting conditions or when individuals are moving quickly. The use of Excel sheets to store attendance data also makes it easy to manage and analyze attendance records, and allows for easy integration with other software tools. The system is able to accurately recognize faces and record attendance in real-time, which is a significant achievement, as attendance tracking is a critical task in many organizations.

Table 6.1: Measuring the Different Parameter Variations

Parameter	Haar cascade classifier	K-Nearest Neighbor
Time	0.01 seconds	0.02 seconds
Space	LESS	MORE
Accuracy	80-90%	95-99%
Robustness	MORE	LESS
Scalability	MORE	LESS

However, there are a few areas where the system could be improved. For example, the system currently relies on a single camera to capture images of individuals. This could be problematic in situations where there are large crowds or where individuals are moving quickly. Additionally, the system could benefit from more advanced machine learning algorithms that are able to adapt to changes in lighting and other environmental factors.

In conclusion, face recognition based attendance system using Haar Cascade algorithm and Excel sheets to get the data of attendance is a promising development in the field of attendance tracking. The system is well-designed and performs admirably in terms of accuracy and efficiency. The use of Haar Cascade algorithm for face detection and Excel sheets to store attendance data is a practical and efficient solution. However, there is still room for improvement, particularly in the areas of scalability and adaptability.

REFERENCES:

1. Prabhavathi, B., V. Tanuja, V. Madhu Viswanatham, and M. Rajashekhara Babu. "A smart technique for attendance system to recognize faces through parallelism." In *IOP Conference Series: Materials Science and Engineering*, vol. 263, no. 4, p. 042095. IOP Publishing, 2017.
2. Bah, Serign Modou, and Fang Ming. "An improved face recognition algorithm and its application in attendance management system." *Array* 5 (2020): 100014.
3. Balcoh, Naveed Khan, M. Haroon Yousaf, Waqar Ahmad, and M. Iram Baig. "Algorithm for efficient attendance management: Face recognition based approach." *International Journal of Computer Science Issues (IJCSI)* 9, no. 4 (2012): 146.
4. D'Souza, Jenif WS, S. Jothi, and A. Chandrasekar. "Automated attendance marking and management system by facial recognition using histogram." In *2019 5th International Conference on Advanced Computing & Communication Systems (ICACCS)*, pp. 66-69. IEEE, 2019.
5. Akbar, Md Sajid, Pronob Sarker, Ahmad Tamim Mansoor, Abu Musa Al Ashray, and Jia Uddin. "Face recognition and RFID verified attendance system." In *2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE)*, pp. 168-172. IEEE, 2018.
6. Wagh, Priyanka, Roshani Thakare, Jagruti Chaudhari, and Shweta Patil. "Attendance system based on face recognition using eigen face and PCA algorithms." In *2015 International Conference on Green Computing and Internet of Things (ICGCIoT)*, pp. 303-308. IEEE, 2015.
7. Kar, Nirmalya, Mrinal Kanti Debbarma, Ashim Saha, and Dwijen Rudra Pal. "Study of implementing automated attendance system using face recognition technique." *International Journal of computer and communication engineering* 1, no. 2 (2012): 100-103.
8. Dev, Samridhi, and Tushar Patnaik. "Student attendance system using face recognition." In *2020 international conference on smart electronics and communication (ICOSEC)*, pp. 90-96. IEEE, 2020.
9. Shah, Kunjal, Dhanashree Bhandare, and Sunil Bhirud. "Face recognition-based automated attendance system." In *International Conference on Innovative Computing and Communications: Proceedings of ICICC 2020, Volume 1*, pp. 945-952. Springer Singapore, 2021.
10. Gomes, Clyde, Sagar Chanchal, Tanmay Desai, and Dipti Jadhav. "Class attendance management system using facial recognition." In *ITM Web of Conferences*, vol. 32, p. 02001. EDP Sciences, 2020.
11. Hossen, Abdul Mohsen Abdul, Raheem Abd Alsaheb Ogla, and Maitham Mahmood Ali. "Face detection by using OpenCV's Viola-Jones algorithm based on coding eyes." *Iraqi Journal of Science* 58, no. 2A (2017): 735-745.
12. Lakshmi, N. Dhana, Pranavi Nagubandi, Muralidhar Yeleti, and K. Vishnu Vardan. "Automated Attendance System Based on Facial Recognition using Viola-Jones Algorithm." *Asian Journal of Applied Science and Technology (AJAST)* 6, no. 2 (2022): 82-91.
13. Yamini, K., K. Mohan, P. Sonia, V. Yugandhar, and K. Bharath. "Class attendance using face detection and recognition with opencv." *International Research Journal of Engineering and Technology* 6, no. 4 (2019): 3822-3825.
14. Dabhade, Sarala A., and Mrunal S. Bewoor. "Real time face detection and recognition using haar-based cascade classifier and principal component analysis." *International Journal of Computer Science and Management Research* 1, no. 1 (2012): 1-5.
15. Joshi, Prateek, David Millán Escrivá, and Vinicius Godoy. *OpenCV By Example*. Packt Publishing Ltd, 2016.
16. Guo, Lei, and Qiu-guang Wang. "Research of face detection based on Adaboost algorithm and OpenCV implementation." *Journal of Harbin University of science and Technology* 14, no. 5 (2009): 123-126.
17. Chintalapati, Shireesha, and M. V. Raghunadh. "Automated attendance management system based on face recognition algorithms." In *2013 IEEE International Conference on Computational Intelligence and Computing Research*, pp. 1-5. IEEE, 2013.
18. Rathod, Hemantkumar, Yudhisthir Ware, Snehal Sane, Suresh Raulo, Vishal Pakhare, and Imdad A. Rizvi. "Automated attendance system using machine learning approach." In *2017 International Conference on Nascent Technologies in Engineering (ICNTE)*, pp. 1-5. IEEE, 2017.
19. Shanthi, S., K. Nirmaladevi, M. Pyingkodi, and P. Selvapandiyan. "Face recognition for automated attendance system using lbph algorithm." *Journal of Critical Reviews* 7, no. 4 (2020): 942-949.
20. Nguyen, Duy Dieu, Xuan Huy Nguyen, The Tung Than, and Minh Son Nguyen. "Automated Attendance System in the Classroom Using Artificial Intelligence and Internet of Things Technology." In *2021 8th NAFOSTED Conference on Information and Computer Science (NICS)*, pp. 531-536. IEEE, 2021.

APPENDIX:

```
##### IMPORTING
```

```
#####
```

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox as mess
import tkinter.simpledialog as tsd
import cv2,os
import csv
import numpy as np
from PIL import Image
import pandas as pd
import datetime
import time
```

```
##### FUNCTIONS
```

```
#####
```

```
def assure_path_exists(path):
    dir = os.path.dirname(path)
    if not os.path.exists(dir):
        os.makedirs(dir)
```

```
#####
```

```
def tick():
    time_string = time.strftime('%H:%M:%S')
    clock.config(text=time_string)
    clock.after(200,tick)
```

```
#####
```

```
def contact():
```

```
mess._show(title='Contact us', message="Please contact us on : 'vignan.ac.in' ")
```

```
#####
```

```
def check_haarcascade():
```

```
    exists = os.path.isfile("haarcascade_frontalface_default.xml")
```

```
    if exists:
```

```
        pass
```

```
    else:
```

```
        mess._show(title='Some file missing', message='Please contact us for help')
```

```
        window.destroy()
```

```
#####
```

```
def save_pass():
```

```
    assure_path_exists("TrainingImageLabel/")
```

```
    exists1 = os.path.isfile("TrainingImageLabel\psd.txt")
```

```
    if exists1:
```

```
        tf = open("TrainingImageLabel\psd.txt", "r")
```

```
        key = tf.read()
```

```
    else:
```

```
        master.destroy()
```

```
        new_pas = tsd.askstring('Old Password not found', 'Please enter a new password below',  
show='*')
```

```
        if new_pas == None:
```

```
            mess._show(title='No Password Entered', message='Password not set!! Please try again')
```

```
        else:
```

```
            tf = open("TrainingImageLabel\psd.txt", "w")
```

```
            tf.write(new_pas)
```

```
            mess._show(title='Password Registered', message='New password was registered  
successfully!!')
```

```
            return
```

```
    op = (old.get())
```

```
    newp= (new.get())
```

```

nnewp = (nnew.get())
if (op == key):
    if(newp == nnewp):
        txf = open("TrainingImageLabel\psd.txt", "w")
        txf.write(newp)
    else:
        mess._show(title='Error', message='Confirm new password again!!!')
        return
else:
    mess._show(title='Wrong Password', message='Please enter correct old password.')
    return
mess._show(title='Password Changed', message='Password changed successfully!!')
master.destroy()

```

#####

```

def change_pass():
    global master
    master = tk.Tk()
    master.geometry("400x160")
    master.resizable(False,False)
    master.title("Change Password")
    master.configure(background="white")
    lbl4 = tk.Label(master,text='  Enter Old Password',bg='white',font=('times', 12, ' bold '))
    lbl4.place(x=10,y=10)
    global old
    old=tk.Entry(master,width=25 ,fg="black",relief='solid',font=('times', 12, ' bold '),show='*')
    old.place(x=180,y=10)
    lbl5 = tk.Label(master, text='  Enter New Password', bg='white', font=('times', 12, ' bold '))
    lbl5.place(x=10, y=45)
    global new
    new = tk.Entry(master, width=25, fg="black",relief='solid', font=('times', 12, ' bold '),show='*')
    new.place(x=180, y=45)
    lbl6 = tk.Label(master, text='Confirm New Password', bg='white', font=('times', 12, ' bold '))

```

```

lbl6.place(x=10, y=80)
global nnew
nnew = tk.Entry(master, width=25, fg="black", relief='solid',font=('times', 12, ' bold
'),show='*')
nnew.place(x=180, y=80)
cancel=tk.Button(master,text="Cancel", command=master.destroy ,fg="black" ,bg="red"
,height=1,width=25 , activebackground = "white" ,font=('times', 10, ' bold '))
cancel.place(x=200, y=120)
save1 = tk.Button(master, text="Save", command=save_pass, fg="black", bg="#3ece48",
height = 1,width=25, activebackground="white", font=('times', 10, ' bold '))
save1.place(x=10, y=120)
master.mainloop()

#####
#

def psw():
    assure_path_exists("TrainingImageLabel/")
    exists1 = os.path.isfile("TrainingImageLabel\psd.txt")
    if exists1:
        tf = open("TrainingImageLabel\psd.txt", "r")
        key = tf.read()
    else:
        new_pas = tsd.askstring('Old Password not found', 'Please enter a new password below',
show='*')
        if new_pas == None:
            mess._show(title='No Password Entered', message='Password not set!! Please try again')
        else:
            tf = open("TrainingImageLabel\psd.txt", "w")
            tf.write(new_pas)
            mess._show(title='Password Registered', message='New password was registered
successfully!!')
            return
        password = tsd.askstring('Password', 'Enter Password', show='*')

```

```

if (password == key):
    TrainImages()
elif (password == None):
    pass
else:
    mess._show(title='Wrong Password', message='You have entered wrong password')

#####
##

def clear():
    txt.delete(0, 'end')
    res = "1)Take Images >>> 2)Save Profile"
    message1.configure(text=res)

def clear2():
    txt2.delete(0, 'end')
    res = "1)Take Images >>> 2)Save Profile"
    message1.configure(text=res)

#####
###

def TakeImages():
    check_haarcascade()
    columns = ['SERIAL NO.', '', 'ID', '', 'NAME']
    assure_path_exists("StudentDetails/")
    assure_path_exists("TrainingImage/")
    serial = 0
    exists = os.path.isfile("StudentDetails\StudentDetails.csv")
    if exists:
        with open("StudentDetails\StudentDetails.csv", 'r') as csvFile1:
            reader1 = csv.reader(csvFile1)

```

```

        for l in reader1:
            serial = serial + 1
        serial = (serial // 2)
        csvFile1.close()
    else:
        with open("StudentDetails\\StudentDetails.csv", 'a+') as csvFile1:
            writer = csv.writer(csvFile1)
            writer.writerow(columns)
            serial = 1
        csvFile1.close()
    Id = (txt.get())
    name = (txt2.get())
    if ((name.isalpha()) or (' ' in name)):
        cam = cv2.VideoCapture(0)
        harcascadePath = "haarcascade_frontalface_default.xml"
        detector = cv2.CascadeClassifier(harcascadePath)
        sampleNum = 0
        while (True):
            ret, img = cam.read()
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            faces = detector.detectMultiScale(gray, 1.3, 5)
            for (x, y, w, h) in faces:
                cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)
                sampleNum = sampleNum + 1
                cv2.imwrite("TrainingImage\\ " + name + "." + str(serial) + "." + Id + '.' +
str(sampleNum) + ".jpg",
                    gray[y:y + h, x:x + w])
                cv2.imshow('Taking Images', img)
            if cv2.waitKey(100) & 0xFF == ord('q'):
                break
            elif sampleNum > 100:
                break
        cam.release()
        cv2.destroyAllWindows()

```

```

res = "Images Taken for ID : " + Id
row = [serial, "", Id, "", name]
with open('StudentDetails\\StudentDetails.csv', 'a+') as csvFile:
    writer = csv.writer(csvFile)
    writer.writerow(row)
csvFile.close()
message1.configure(text=res)
else:
    if (name.isalpha() == False):
        res = "Enter Correct name"
        message.configure(text=res)

#####

####

def getImagesAndLabels(path):
    imagePaths = [os.path.join(path, f) for f in os.listdir(path)]
    faces = []
    Ids = []
    for imagePath in imagePaths:
        pilImage = Image.open(imagePath).convert('L')
        imageNp = np.array(pilImage, 'uint8')
        ID = int(os.path.split(imagePath)[-1].split(".")[1])
        faces.append(imageNp)
        Ids.append(ID)
    return faces, Ids

#####

#####

def TrainImages():
    check_haarcascade()

```



```

assure_path_exists("TrainingImageLabel/")
recognizer = cv2.face_LBPHFaceRecognizer.create()
harcascadePath = "haarcascade_frontalface_default.xml"
detector = cv2.CascadeClassifier(harcascadePath)
faces, ID = getImagesAndLabels("TrainingImage")
try:
    recognizer.train(faces, np.array(ID))
except:
    mess._show(title='No Registrations', message='Please Register someone first!!!')
    return
recognizer.save("TrainingImageLabel\Trainer.yml")
res = "Profile Saved Successfully"
message1.configure(text=res)
message.configure(text='Total Registrations till now : ' + str(ID[0]))

#####
#####3

```

```

def TrackImages():
    check_haarcascadefile()
    assure_path_exists("Attendance/")
    assure_path_exists("StudentDetails/")
    for k in tv.get_children():
        tv.delete(k)
    msg = ""
    i = 0
    j = 0
    recognizer = cv2.face.LBPHFaceRecognizer_create() # cv2.createLBPHFaceRecognizer()
    exists3 = os.path.isfile("TrainingImageLabel\Trainer.yml")
    if exists3:
        recognizer.read("TrainingImageLabel\Trainer.yml")
    else:

```

```

    mess._show(title='Data Missing', message='Please click on Save Profile to reset data!!')
    return
harcascadePath = "haarcascade_frontalface_default.xml"
faceCascade = cv2.CascadeClassifier(harcascadePath);

cam = cv2.VideoCapture(0)
font = cv2.FONT_HERSHEY_SIMPLEX
col_names = ['Id', '', 'Name', '', 'Date', '', 'Time']
exists1 = os.path.isfile("StudentDetails\\StudentDetails.csv")
if exists1:
    df = pd.read_csv("StudentDetails\\StudentDetails.csv")
else:
    mess._show(title='Details Missing', message='Students details are missing, please check!')
    cam.release()
    cv2.destroyAllWindows()
    window.destroy()
while True:
    ret, im = cam.read()
    gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
    faces = faceCascade.detectMultiScale(gray, 1.2, 5)
    for (x, y, w, h) in faces:
        cv2.rectangle(im, (x, y), (x + w, y + h), (225, 0, 0), 2)
        serial, conf = recognizer.predict(gray[y:y + h, x:x + w])
        if (conf < 50):
            ts = time.time()
            date = datetime.datetime.fromtimestamp(ts).strftime('%D-%m-%Y')
            timeStamp = datetime.datetime.fromtimestamp(ts).strftime('%H:%M:%S')
            aa = df.loc[df['SERIAL NO.'] == serial]['NAME'].values
            ID = df.loc[df['SERIAL NO.'] == serial]['ID'].values
            ID = str(ID)
            ID = ID[1:-1]
            bb = str(aa)
            bb = bb[2:-2]
            attendance = [str(ID), "", bb, "", str(date), "", str(timeStamp)]

```

```

else:
    Id = 'Unknown'
    bb = str(Id)
    cv2.putText(im, str(bb), (x, y + h), font, 1, (255, 255, 255), 2)
cv2.imshow('Taking Attendance', im)
if (cv2.waitKey(1) == ord('q')):
    break
ts = time.time()
date = datetime.datetime.fromtimestamp(ts).strftime('%d-%m-%Y')
exists = os.path.isfile("Attendance\Attendance_" + date + ".csv")
if exists:
    with open("Attendance\Attendance_" + date + ".csv", 'a+') as csvFile1:
        writer = csv.writer(csvFile1)
        writer.writerow(attendance)
    csvFile1.close()
else:
    with open("Attendance\Attendance_" + date + ".csv", 'a+') as csvFile1:
        writer = csv.writer(csvFile1)
        writer.writerow(col_names)
        writer.writerow(attendance)
    csvFile1.close()
with open("Attendance\Attendance_" + date + ".csv", 'r') as csvFile1:
    reader1 = csv.reader(csvFile1)
    for lines in reader1:
        i = i + 1
        if (i > 1):
            if (i % 2 != 0):
                iidd = str(lines[0]) + ' '
                tv.insert(' ', 0, text=iidd, values=(str(lines[2]), str(lines[4]), str(lines[6])))
csvFile1.close()
cam.release()
cv2.destroyAllWindows()

```

```
##### USED STUFFS
```

```
#####
```

```
global key
```

```
key = ''
```

```
ts = time.time()
```

```
date = datetime.datetime.fromtimestamp(ts).strftime('%d-%m-%Y')
```

```
day,month,year=date.split("-")
```

```
mont={'01':'January',
```

```
      '02':'February',
```

```
      '03':'March',
```

```
      '04':'April',
```

```
      '05':'May',
```

```
      '06':'June',
```

```
      '07':'July',
```

```
      '08':'August',
```

```
      '09':'September',
```

```
      '10':'October',
```

```
      '11':'November',
```

```
      '12':'December'
```

```
  }
```

```
##### GUI FRONT-END
```

```
#####
```

```
window = tk.Tk()
```

```
window.geometry("1280x720")
```

```
window.resizable(True, False)
```

```
window.title("Vignan's Attendance System")
```

```
window.configure(background='#000000')
```

```

frame1 = tk.Frame(window, bg="#000000")
frame1.place(relx=0.11, rely=0.17, relwidth=0.39, relheight=0.80)

frame2 = tk.Frame(window, bg="#000000")
frame2.place(relx=0.51, rely=0.17, relwidth=0.38, relheight=0.80)

message3 = tk.Label(window, text="Face Recognition Based Attendance System",
                    fg="#d3d3d3", bg="#000000", width=55, height=1, font=('calibri', 24, 'bold'))
message3.place(x=190, y=10)

frame3 = tk.Frame(window, bg="#000000")
frame3.place(relx=0.52, rely=0.09, relwidth=0.09, relheight=0.07)

frame4 = tk.Frame(window, bg="#000000")
frame4.place(relx=0.36, rely=0.09, relwidth=0.16, relheight=0.07)

datef = tk.Label(frame4, text=day+"-"+mont[month]+"-"+year+" | ", fg="#808080",
                    bg="#000000", width=55, height=1, font=('calibri', 14, 'bold'))
datef.pack(fill='both', expand=1)

clock = tk.Label(frame3, fg="#808080", bg="#000000", width=55,
                    height=1, font=('calibri', 14, 'bold'))
clock.pack(fill='both', expand=1)
tick()

head2 = tk.Label(frame2, text="                                For New Registrations                                ",
                    fg="#D3D3D3", bg="#3D3D3D", font=('calibri', 17, 'bold'))
head2.grid(row=0, column=0)

head1 = tk.Label(frame1, text="                                For Already Registered                                ",
                    fg="#D3D3D3", bg="#3D3D3D", font=('calibri', 17, 'bold'))
head1.place(x=0, y=0)

lbl = tk.Label(frame2, text="Enter ID", width=0, height=1,

```

```

        fg="#D3D3D3", bg="#000000", font=('calibri', 14, ' bold '))
lbl.place(x=26, y=55)

txt = tk.Entry(frame2, width=32, fg="black", font=('calibri', 15, ' bold '))
txt.place(x=50, y=86)

lbl2 = tk.Label(frame2, text="Enter Name", width=0, fg="#D3D3D3",
                bg="#000000", font=('calibri', 14, ' bold '))
lbl2.place(x=26, y=140)

txt2 = tk.Entry(frame2, width=32, fg="black", font=('calibri', 15, ' bold '))
txt2.place(x=50, y=173)

message1 = tk.Label(frame2, text="1)Take Images >>> 2)Save Profile",bg="#000000"
,fg="#D3D3D3" ,width=39 ,height=1, activebackground = "yellow" ,font=('times', 15, ' bold '))
message1.place(x=7, y=230)

message = tk.Label(frame2, text="", bg="#3D3D3D", fg="#D3D3D3", width=39,
                height=1, activebackground="yellow", font=('calibri', 16, ' bold '))
message.place(x=35, y=470)

lbl3 = tk.Label(frame1, text="Attendance", width=20, fg="#D3D3D3",
                bg="#000000", height=1, font=('calibri', 16, ' bold '))
lbl3.place(x=130, y=108)

res = 0
exists = os.path.isfile("StudentDetails\\StudentDetails.csv")
if exists:
    with open("StudentDetails\\StudentDetails.csv", 'r') as csvFile1:
        reader1 = csv.reader(csvFile1)
        for l in reader1:
            res = res + 1
res = (res // 2) - 1

```

```

        csvFile1.close()
    else:
        res = 0
    message.configure(text='Total Registrations till now : '+str(res))

##### MENUBAR #####

menubar = tk.Menu(window,relief='ridge')
filemenu = tk.Menu(menubar,tearoff=0)
filemenu.add_command(label='Change Password', command = change_pass)
filemenu.add_command(label='Contact Us', command = contact)
filemenu.add_command(label='Exit',command = window.destroy)
menubar.add_cascade(label='Help',font=('times', 29, ' bold '),menu=filemenu)

##### TREEVIEW ATTENDANCE TABLE #####

tv = ttk.Treeview(frame1, height=13, columns=('name', 'date', 'time'))
tv.column('#0', width=85)
tv.column('name', width=130)
tv.column('date', width=130)
tv.column('time', width=130)
tv.grid(row=2, column=0, padx=(0, 0), pady=(150, 0), columnspan=4)
tv.heading('#0', text='ID')
tv.heading('name', text='NAME')
tv.heading('date', text='DATE')
tv.heading('time', text='TIME')

##### SCROLLBAR #####

scroll=ttk.Scrollbar(frame1,orient='vertical',command=tv.yview)
scroll.grid(row=2,column=4,padx=(0,100),pady=(150,0),sticky='ns')
tv.configure(yscrollcommand=scroll.set)

```

BUTTONS

```
clearButton = tk.Button(frame2, text="Clear", command=clear, fg="black", bg="#ea2a2a",
                        width=11, activebackground="gray", border=0, font=('calibri', 11, ' bold '))
clearButton.place(x=380, y=86)
```

```
clearButton2 = tk.Button(frame2, text="Clear", command=clear2, fg="black",
                        bg="#ea2a2a", width=11, activebackground="gray", border=0, font=('calibri', 11, '
bold '))
clearButton2.place(x=380, y=173)
```

```
takeImg = tk.Button(frame2, text="Take Images", command=TakeImages, fg="black",
                    bg="#57beba",
                        width=34, height=1, activebackground="gray", border=0, font=('calibri', 15, ' bold '))
takeImg.place(x=70, y=290)
```

```
trainImg = tk.Button(frame2, text="Save Profile", command=psw, fg="black", bg="#57beba",
                    width=34, height=1, activebackground="gray", border=0, font=('calibri', 15, ' bold '))
trainImg.place(x=70, y=370)
```

```
trackImg = tk.Button(frame1, text="Take Attendance", command=TrackImages, fg="#000000",
                    bg="#4cbb17", width=35, height=1, activebackground="gray", border=0,
font=('calibri', 15, ' bold '))
trackImg.place(x=65, y=50)
```

```
quitWindow = tk.Button(frame1, text="Quit", command=window.destroy, fg="black", bg="red",
                    width=14, height=1, activebackground="gray", border=0, font=('calibri', 15, ' bold
'))
quitWindow.place(x=160, y=460)
```

END

```
window.configure(menu=menubar)
window.mainloop()
```